Solving CarRacing-v3 with Deep Reinforcement Learning

A Complete Guide to DQN and PPO for Autonomous Racing

Adrien Fu & Raphaël Faure

March 2025

Framework: PyTorch · Gymnasium · Stable-Baselines3

1. Executive Summary

This project implements and compares Deep Reinforcement Learning algorithms to solve the CarRacing-v3 environment from Gymnasium. An autonomous racing car learns to drive on procedurally generated tracks using only raw pixel observations. Two complementary approaches are explored: a custom Deep Q-Network (DQN) built from scratch in PyTorch and a Proximal Policy Optimization (PPO) agent powered by Stable-Baselines3. Through careful preprocessing, architecture design, and hyper-parameter tuning, the PPO agent achieves an average reward of **733+** in roughly **3 hours** of training, while the DQN agent reaches **605** after approximately **12 hours**, clearly demonstrating the sample-efficiency and stability advantages of modern policy-gradient methods for continuous-control-like tasks with visual observations.

---

2. Environment & Observation Pipeline

Environment Specification

- **CarRacing-v3** from Gymnasium – top-down 2D racing on procedurally generated tracks rendered at 96×96 RGB pixels.
- **Action space:** 5 discrete actions (do nothing, steer left, steer right, gas, brake).
- **Physics engine:** Box2D rigid-body simulation.

Preprocessing Pipeline

- **Crop** the 96×96 frame to 60×60 (remove HUD / sky).
- **Grayscale** conversion to reduce dimensionality.
- **Normalize** pixel values to the [0, 1] range.
- **Frame stacking:** concatenate 4 consecutive frames → tensor of shape `[4, 60, 60]`, giving the agent velocity/acceleration cues.

Reward Structure

- +1000 / N per new track tile visited (N = total tiles).
- –0.1 per frame (encourages speed).
- –100 penalty when driving off-track.

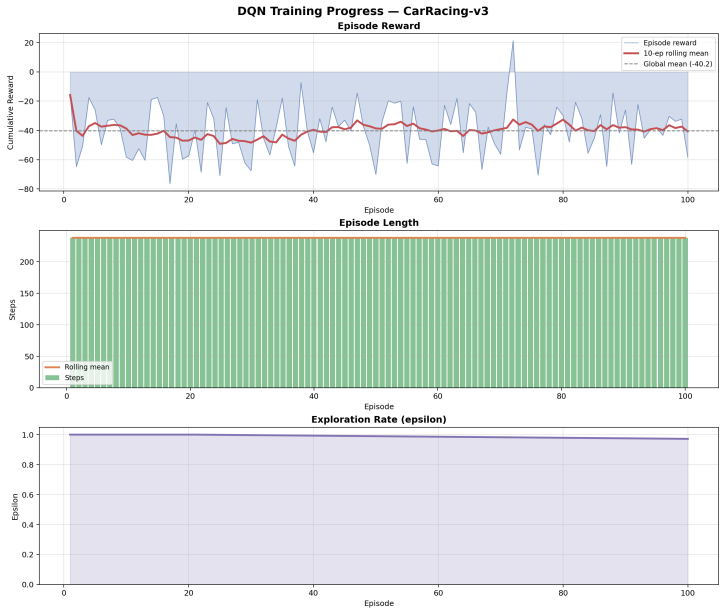


Figure 1 — Training progress over episodes.

3. DQN Algorithm

A fully custom Deep Q-Network implementation in PyTorch, trained from scratch without external RL libraries.

Network Architecture

```
Conv2d(4→16, 8×8, stride 4) → ReLU
Conv2d(16→32, 4×4, stride 2) → ReLU
Flatten → FC(1152→256) → ReLU → FC(256→5)
```

Key Components

- **Experience Replay Buffer** — stores up to 100,000 transitions; uniformly sampled mini-batches of 32.
- **Target Network** — a frozen copy of the Q-network, hard-synced every C steps to stabilise training.
- **Epsilon-Greedy Exploration** — ε decays linearly from 1.0 → 0.1 over 600,000 environment steps.

Hyper-parameters

Parameter	Value
Optimizer	RMSProp
Learning Rate	0.0004

Discount (γ)	0.99
Replay Buffer	100,000
Batch Size	32
Target Sync	Every 1,000 steps
ε Schedule	1.0 → 0.1 (600k steps)

4. PPO Algorithm

Proximal Policy Optimization via Stable-Baselines3, leveraging a shared CNN backbone for both actor and critic.

Architecture & Design

• **CnnPolicy** (Stable-Baselines3 default) — shared convolutional feature extractor feeding separate policy and value heads.

• **Clipped Surrogate Objective** — clip ratio ε = 0.2 prevents destructively large policy updates.

• **Generalized Advantage Estimation (GAE)** — λ = 0.95 balances bias–variance in advantage estimates.

• **Training budget:** 650,000 environment steps, completed in approximately 3 hours on a single GPU.



Figure 2 — Debug trail visualization of resumed training.

5. Results Comparison

The table below summarises the key metrics for each agent after full training runs.

Model	Steps	Time	Final Return	Stability
DQN	600,000	~12 h	605	High variance
PPO	650,000	~3 h	**733**	More stable

PPO achieves a **21 % higher return** in roughly **one quarter** of the training time, confirming the superior sample efficiency of on-policy actor-critic methods for this domain.

---

6. The Learning Studio

`lap_learning_studio.py` is an advanced training visualization and debugging tool built specifically for this project.

- **Whole-lap minimap** — real-time bird's-eye view with the car's position overlaid on the full track.
- **Off-road penalty with instant restart** — detects when the car leaves the track and resets immediately to accelerate learning.
- **Lap timer & finish-line detection** — measures completion time and detects when the agent completes a full lap.
- **Interactive speed control** — keyboard hotkeys allow changing simulation speed during live rendering (1×, 2×, 4×, pause).

---

7. How to Run

Step 1 — Install dependencies

```
python setup.py
```

Step 2 — Interactive notebook

```
jupyter notebook CarRacing_v3_Complete_Guide.ipynb
```

Step 3 — Train an agent

```
python src/main.py --agent PPO
```

Step 4 — Launch the Learning Studio

```
python src/lap_learning_studio.py --render
```

---

8. References

[1] Mnih, V. et al. (2013). *Playing Atari with Deep Reinforcement Learning.* arXiv:1312.5602.

[2] Schulman, J. et al. (2017). *Proximal Policy Optimization Algorithms.* arXiv:1707.06347.

[3] Raffin, A. et al. (2021). *Stable-Baselines3: Reliable Reinforcement Learning Implementations.* JMLR 22(268):1–8.

[4] Farama Foundation (2023). *Gymnasium: A Standard API for Reinforcement Learning.* https://gymnasium.farama.org.